STEP-BY-STEP IMPLEMENTATION GUIDE

Production-Grade Nginx Reverse Proxy

Secure backend · High availability · Monitoring · Incident recovery

Platform
Windows 11 + Docker Desktop (lab)
Ubuntu 22.04/24.04 (production)

Stack
Nginx 1.28, Flask/Gunicorn, Prometheus 2.55, Grafana 11.3

Result
46/46 tests passed
8/8 incident drills recovered

29 September 2026

Contents

1. Overview and architecture
2. Prerequisites
3. Step 1: Infrastructure preparation
4. Step 2: Linux server configuration
5. Step 3: Backend application deployment
6. Step 4: Nginx reverse proxy configuration
7. Step 5: Backend security and firewall
8. Step 6: SSL certificate and HTTPS setup
9. Step 7: High availability and load balancing
10. Step 8: Monitoring and Grafana dashboard
11. Step 9: Incident simulation and recovery
12. Step 10: Testing and documentation
13. Production deployment on Ubuntu servers
14. Command cheat sheet and troubleshooting
15. Results summary

All terminal screenshots in this guide show real output from running the listed commands in Windows PowerShell on the lab machine. Long outputs are shortened; every omission is marked with “… lines omitted …”. Browser screenshots were taken with Chromium against the running lab.

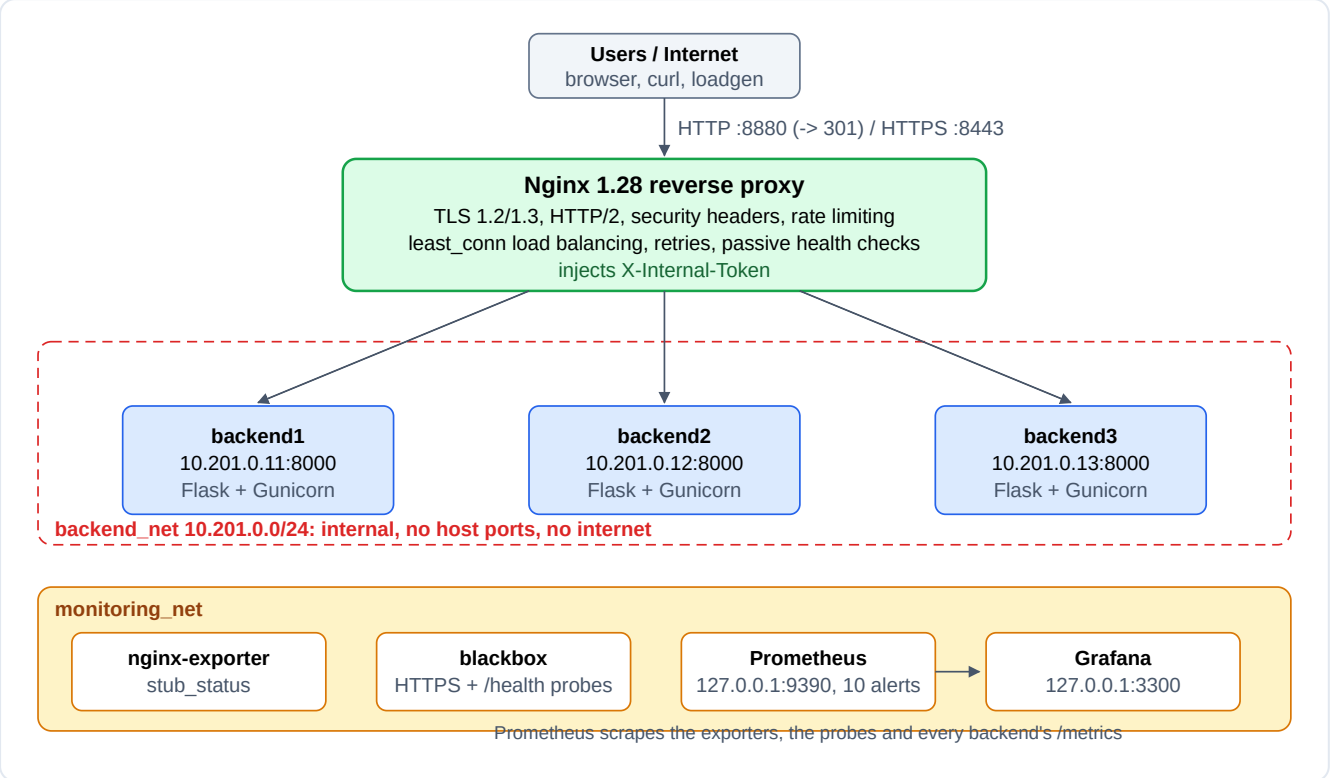
1. Overview and architecture

This project puts an **Nginx reverse proxy** in front of a pool of three backend API servers. Users only ever talk to Nginx over HTTPS. The backends sit on a private network that cannot be reached from outside and only accept requests carrying a secret token that Nginx adds. Nginx spreads traffic across the backends and automatically routes around any that fail. Prometheus and Grafana watch everything and raise alerts, and eight scripted incident drills prove the system recovers.

The project runs in two forms built from the same configuration files:

	Local lab (this guide)	Production
Runs on	One Windows PC with Docker Desktop	6 Ubuntu VMs
Start with	`.\lab.ps1`	`scripts/linux/01..08-*.sh`
Backend isolation	internal Docker network	private subnet + UFW firewall
TLS certificate	private lab CA	Let's Encrypt with auto-renewal
Proxy HA	auto-restart	2 proxies + keepalived floating IP

Lab architecture



Component	Purpose	Address (lab)
Nginx	Reverse proxy, TLS, load balancer	https://localhost:8443, http://localhost:8880
backend1-3	Flask API with metrics	10.201.0.11-13:8000 (internal only)
Prometheus	Metrics + alert rules	http://127.0.0.1:9390
Grafana	Dashboard	http://127.0.0.1:3300
Blackbox exporter	Synthetic HTTPS and health probes	internal
nginx-exporter	Nginx connection metrics	internal

> **Tip:** The lab uses ports 8880/8443/3300/9390 instead of 80/443/3000/9090 so it can run next to other Docker projects on the same machine. Change them in `.env` if needed.

2. Prerequisites

Software	Why	Get it
Windows 10/11	host OS	–
Docker Desktop	runs all containers	docker.com/products/docker-desktop
Git for Windows	provides Git Bash, used behind the scenes by `lab.ps1`, plus OpenSSL	git-scm.com/download/win
~4 GB free RAM, ~3 GB disk	images and containers	–

Open **Windows PowerShell** and check that the tools are installed:

PowerShell

```
docker --version
docker compose version
git --version
```

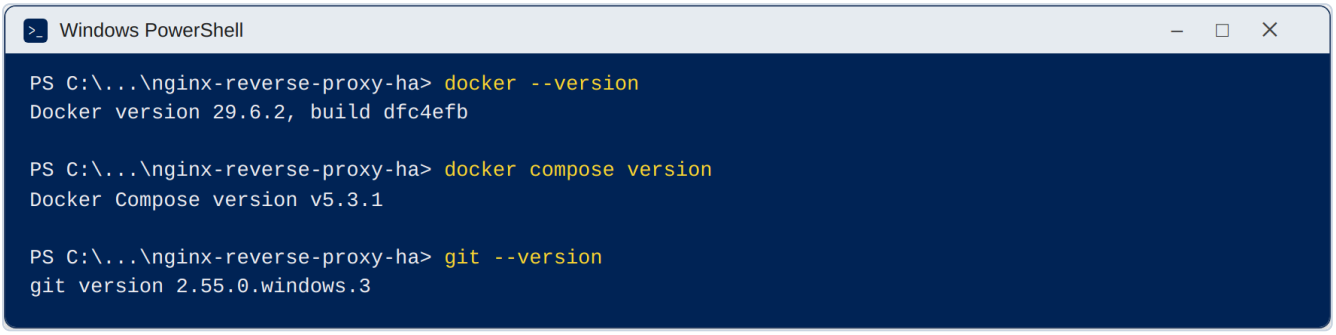


Figure 1: Docker, Docker Compose and Git are installed

> **Important:** Docker Desktop must be running (whale icon in the system tray) before any lab command.

3. Step 1: Infrastructure preparation

In the lab, “infrastructure” is defined in `docker-compose.yml` : three isolated networks (frontend, backend, monitoring), fixed IP addresses for the backends, resource limits and restart policies. Nothing has to be provisioned by hand.

1.1 Open the project folder

PowerShell

```
cd "C:\Users\kishwanth.aderu\Desktop\Claude Projects\nginx-reverse-proxy-ha"
```

1.2 Allow local scripts in this PowerShell window

This only affects the current window and does not change any system setting. Repeat it whenever you open a new window.

PowerShell

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force
```

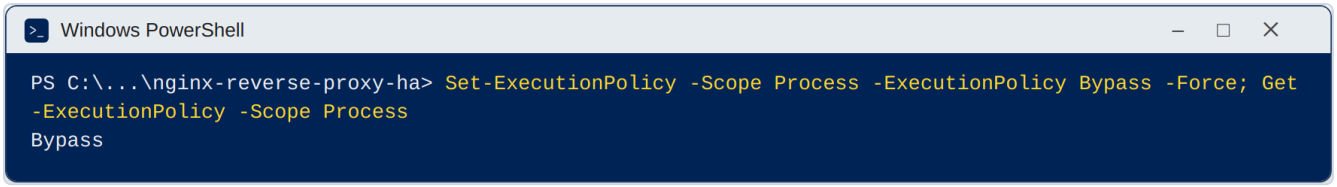


Figure 2: Execution policy set for the current PowerShell session

1.3 Project layout

Folder / file	Contents
`lab.ps1` , `lab.sh`	lab controller (PowerShell wrapper and the bash script it runs)
`docker-compose.yml`	networks, services, IP plan, security options
`backend/`	Flask API, Dockerfile, Gunicorn config
`nginx/`	nginx.conf, site template, snippets (TLS, headers, proxy), error page
`monitoring/`	Prometheus config + alert rules, Blackbox, Grafana dashboard
`incidents/`	8 incident drills
`tests/`	46 acceptance tests, production config validation
`scripts/linux/`	production scripts for phases 1-8 on Ubuntu
`docs/`	architecture, runbook, deployment guide, test report

1.4 IP plan (lab)

Network	Subnet	Members	Reachable from host?
frontend_net	Docker-assigned	nginx, loadgen	yes: ports 8880, 8443
backend_net	10.201.0.0/24 (internal)	nginx .10, backends .11-.13, prometheus, blackbox	no
monitoring_net	Docker-assigned	nginx, exporters, prometheus, grafana	only 127.0.0.1:3300 / 9390

4. Step 2: Linux server configuration

In the lab every service runs in a hardened Linux container. The same hardening appears in `docker-compose.yml` and `backend/Dockerfile`:

- backends run as an unprivileged user (uid 10001), never root
- read-only root filesystem, all Linux capabilities dropped, `no-new-privileges`
- CPU and memory limits (0.5 CPU, 256 MB per backend)
- log rotation (10 MB x 3 files) and automatic restart

Verification is part of Step 5 (Figure showing `uid=10001(app)`).

On real Ubuntu servers

The production script configures the operating system on every host: packages, time sync, admin user, SSH hardening (password login disabled only once a key is installed), kernel tuning, automatic security updates, fail2ban and node_exporter.

bash (Ubuntu server)

```
sudo scripts/linux/02-server-config.sh lb           # on lb1, lb2
sudo scripts/linux/02-server-config.sh backend      # on app1, app2, app3
sudo scripts/linux/02-server-config.sh monitoring   # on mon
```

> **Note:** Production commands are shown for completeness. They were not executed as part of this lab; their generated configurations were validated in Step 10.

5. Step 3: Backend application deployment

The backend is a small Flask API served by Gunicorn. It exposes `/`, `/api/info`, `/health` and `/metrics` (Prometheus). Three copies run behind Nginx.

3.1 First-time setup (build and start everything)

This single command creates `.env` with random secrets, generates the TLS certificates, builds the backend image and starts all 8 containers, waiting until they are healthy.

PowerShell

```
.\lab.ps1 setup
```

On later runs, start the stack with:

PowerShell

```
.\lab.ps1 up
```

Windows PowerShell

```
PS C:\...\nginx-reverse-proxy-ha> .\lab.ps1 up
… docker build output omitted …
 Container nginx-ha-lab-nginx-exporter-1 Healthy
 Container nginx-ha-lab-grafana-1 Healthy
 Container nginx-ha-lab-nginx-1 Healthy
 Container nginx-ha-lab-blackbox-1 Healthy
 Container nginx-ha-lab-prometheus-1 Healthy
 Container nginx-ha-lab-backend2-1 Healthy
 Container nginx-ha-lab-backend3-1 Healthy
 Container nginx-ha-lab-backend1-1 Healthy
[ OK ] stack is up and healthy
```

Figure 3: Stack built and started; every container reports healthy

3.2 Check the status

PowerShell

```
.\lab.ps1 status
```

Windows PowerShell

```
PS C:\...\nginx-reverse-proxy-ha> .\lab.ps1 status

==> Containers
SERVICE          STATUS                    PORTS
backend1         Up 9 seconds (healthy)    8000/tcp
backend2         Up 9 seconds (healthy)    8000/tcp
backend3         Up 9 seconds (healthy)    8000/tcp
blackbox         Up 2 hours                9115/tcp
grafana          Up 2 hours                127.0.0.1:3300->3000/tcp
nginx            Up 16 minutes (healthy)   0.0.0.0:8880->80/tcp, [::]:8880->80/tcp, 0.0.0.0:8443->443/tc
p, [::]:8443->443/tcp
nginx-exporter   Up 2 hours
prometheus       Up 2 hours                127.0.0.1:9390->9090/tcp

==> Backend health (Prometheus blackbox probes)
  backend1  UP
  backend2  UP
  backend3  UP

==> End-to-end HTTPS
  GET /nginx-health -> 200
  GET /api/info     -> 200

==> Alerts (pending/firing)
    none
```

Figure 4: All containers up, all three backends healthy, end-to-end HTTPS returns 200, no alerts

3.3 Call the API through the proxy



Figure 5: The API answered through Nginx at https://app.local/api/info (backend name shown)

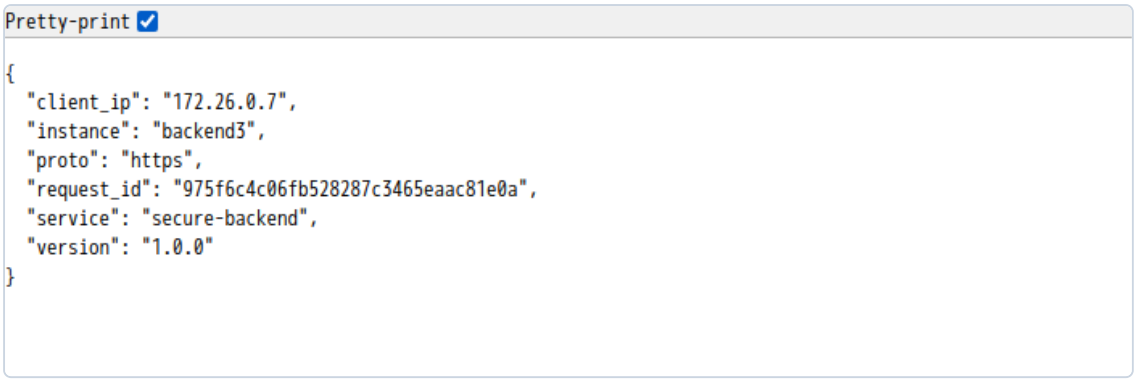


Figure 6: Root endpoint: the backend sees the real client IP, request ID and https as the protocol

6. Step 4: Nginx reverse proxy configuration

The Nginx configuration is split into small files:

File	What it does
`nginx/nginx.conf`	workers, hidden version, timeouts, gzip, JSON access log, rate-limit zones
`nginx/templates/app.conf.template`	upstream pool, HTTP->HTTPS redirect, TLS server, blocked paths, status server
`nginx/snippets/ssl-params.conf`	TLS 1.2/1.3, modern ciphers, session cache
`nginx/snippets/security-headers.conf`	HSTS, CSP, X-Frame-Options, nosniff, Referrer-Policy
`nginx/snippets/proxy-params.conf`	forwarded headers, timeouts, retry/failover rules

4.1 The upstream (load-balancing pool)

nginx

```
upstream backend_pool {
    zone backend_pool 64k;
    least_conn;
    server 10.201.0.11:8000 max_fails=3 fail_timeout=10s;   # backend1
    server 10.201.0.12:8000 max_fails=3 fail_timeout=10s;   # backend2
    server 10.201.0.13:8000 max_fails=3 fail_timeout=10s;   # backend3
    keepalive 32;
}
```

4.2 Validate the configuration

PowerShell

```
docker compose exec -T nginx nginx -t
docker compose exec -T nginx nginx -v
```

Windows PowerShell

```
PS C:\...\nginx-reverse-proxy-ha> docker compose exec -T nginx nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful

PS C:\...\nginx-reverse-proxy-ha> docker compose exec -T nginx nginx -v
nginx version: nginx/1.28.3
```

Figure 7: Configuration test passes; Nginx 1.28.3

4.3 Request through the proxy

PowerShell

```
curl.exe -s -i --ssl-no-revoke --cacert certs\ca.crt https://localhost:8443/api/info
```

Windows PowerShell

```
PS C:\...\nginx-reverse-proxy-ha> curl.exe -s -i --ssl-no-revoke --cacert certs\ca.crt https://localhos
t:8443/api/info
HTTP/1.1 200 OK
Server: nginx
Date: Tue, 29 Sep 2026 12:57:38 GMT
Content-Type: application/json
Content-Length: 64
Connection: keep-alive
X-Backend-Instance: backend3
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: geolocation=(), microphone=(), camera=()
Content-Security-Policy: default-src 'self'; frame-ancestors 'none'; base-uri 'self'
X-Request-ID: 3d72a2ad252bf7bb9387486a445b9d91

{"instance":"backend3","uptime_seconds":10.2,"version":"1.0.0"}
```

Figure 8: HTTP 200 from a backend, with the security headers added by Nginx

4.4 HTTP is redirected to HTTPS

PowerShell

```
curl.exe -s -I http://localhost:8880/api/info
```

Windows PowerShell

```
PS C:\...\nginx-reverse-proxy-ha> curl.exe -s -I http://localhost:8880/api/info
HTTP/1.1 301 Moved Permanently
Server: nginx
Date: Tue, 29 Sep 2026 12:57:39 GMT
Content-Type: text/html
Content-Length: 162
Connection: keep-alive
Location: https://localhost:8443/api/info
```

Figure 9: Plain HTTP gets a permanent 301 redirect to HTTPS, keeping the path

7. Step 5: Backend security and firewall

The backends are protected by several independent layers:

1. **Network isolation**: backends live on an `internal` network with no published ports and no internet access.
2. **Shared secret**: the backend rejects any request without the `X-Internal-Token` header that only Nginx adds (403).
3. **Edge filtering**: Nginx blocks `/metrics`, `/admin/*`, dotfiles and unusual methods such as TRACE.
4. **Least privilege**: non-root user, read-only filesystem.

5.1 Prove the backend is isolated

PowerShell

```
curl.exe -s -m 3 http://localhost:8000/; "exit code: $LASTEXITCODE"
docker network inspect nginx-ha-lab_backend_net --format "internal={{.Internal}}  subnet={{(index .IPAM.Confi
g 0).Subnet}}"
docker compose exec -T nginx wget -q -S -O - http://10.201.0.11:8000/api/info
docker compose exec -T backend1 id
```

Windows PowerShell

```
PS C:\...\nginx-reverse-proxy-ha> curl.exe -s -m 3 http://localhost:8000/; "exit code: $LASTEXITCODE  (b
ackend port is not published)"
exit code: 7  (backend port is not published)

PS C:\...\nginx-reverse-proxy-ha> docker network inspect nginx-ha-lab_backend_net --format "internal={{.
Internal}}  subnet={{(index .IPAM.Config 0).Subnet}}"
internal=true  subnet=10.201.0.0/24

PS C:\...\nginx-reverse-proxy-ha> docker compose exec -T nginx wget -q -S -O - http://10.201.0.11:8000/a
pi/info
  HTTP/1.1 403 FORBIDDEN
wget: server returned error: HTTP/1.1 403 FORBIDDEN

PS C:\...\nginx-reverse-proxy-ha> docker compose exec -T backend1 id
uid=10001(app) gid=10001(app) groups=10001(app)
```

Figure 10: Port 8000 is unreachable from the host; the backend network is internal; a direct call without the token gets 403; the backend runs as uid 10001

5.2 Internal paths and dangerous methods are blocked at the edge

PowerShell

```
foreach ($p in "/metrics","/admin/chaos","/.env") { curl.exe -s --ssl-no-revoke --cacert certs\ca.crt -o NUL -w "GET $p -> %{http_code}`n" "https://localhost:8443$p" }
curl.exe -s --ssl-no-revoke --cacert certs\ca.crt -o NUL -w "TRACE / -> %{http_code}`n" -X TRACE https://localhost:8443/
```

Windows PowerShell

```
PS C:\...\nginx-reverse-proxy-ha> foreach ($p in "/metrics","/admin/chaos","/.env") { curl.exe -s --ssl-no-revoke --cacert certs\ca.crt -o NUL -w "GET $p -> %{http_code}`n" "https://localhost:8443$p" }
GET /metrics -> 404
GET /admin/chaos -> 404
GET /.env -> 404

PS C:\...\nginx-reverse-proxy-ha> curl.exe -s --ssl-no-revoke --cacert certs\ca.crt -o NUL -w "TRACE / -> %{http_code}`n" -X TRACE https://localhost:8443/
TRACE / -> 405
```

Figure 11: /metrics, /admin and dotfiles return 404; TRACE returns 405

5.3 Security headers

PowerShell

```
curl.exe -s -I --ssl-no-revoke --cacert certs\ca.crt https://localhost:8443/
```

Windows PowerShell

```
PS C:\...\nginx-reverse-proxy-ha> curl.exe -s -I --ssl-no-revoke --cacert certs\ca.crt https://localhost:8443/
HTTP/1.1 200 OK
Server: nginx
Date: Tue, 29 Sep 2026 12:58:01 GMT
Content-Type: application/json
Content-Length: 158
Connection: keep-alive
X-Backend-Instance: backend2
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: geolocation=(), microphone=(), camera=()
Content-Security-Policy: default-src 'self'; frame-ancestors 'none'; base-uri 'self'
X-Request-ID: 5a8e5aad6a30c44c58d142573983de36
```

Figure 12: Security headers on every response; the Server header hides the Nginx version

> **Note:** HSTS is deliberately not sent for `localhost`: browsers apply HSTS per host name, so it would force HTTPS on every other localhost project. It is sent for real names (app.local, production domain).

Production firewall (UFW)

Port	lb1 / lb2	app1-3	mon
22 SSH	ADMIN_CIDR	ADMIN_CIDR	ADMIN_CIDR
80 / 443	anywhere	–	–
8000 backend	–	lb1, lb2, mon only	–
9100 / 9113 exporters	mon only	mon only	–
VRRP (proto 112)	peer LB only	–	–
3000 / 9090	–	–	ADMIN_CIDR (DOCKER-USER rule too)

bash (Ubuntu server)

```
sudo scripts/linux/05-firewall.sh lb
sudo scripts/linux/05-firewall.sh backend        # optional: --strict-egress
sudo scripts/linux/05-firewall.sh monitoring
```

8. Step 6: SSL certificate and HTTPS setup

The lab uses its own certificate authority (`certs/ca.crt`) to sign a server certificate for `app.local` , `localhost` and `127.0.0.1` . It is created automatically by `.\lab.ps1 setup` . Nginx allows only TLS 1.2 and 1.3 with modern ciphers, negotiates HTTP/2, and refuses the TLS handshake for unknown host names.

6.1 Verify the TLS setup

PowerShell

```
echo "" | openssl s_client -connect 127.0.0.1:8443 -servername app.local -CAfile certs\ca.crt -brief 2>&1 | S
elect-String "Protocol|Ciphersuite|Verification|Peer certificate"
echo "" | openssl s_client -connect 127.0.0.1:8443 -servername app.local -tls1_1 -cipher "DEFAULT@SECLEVEL=0"
2>&1 | Select-String "alert|error" | Select-Object -First 1
echo "" | openssl s_client -connect 127.0.0.1:8443 -servername app.local -alpn h2,http/1.1 2>&1 | Select-Stri
ng "ALPN protocol"
```

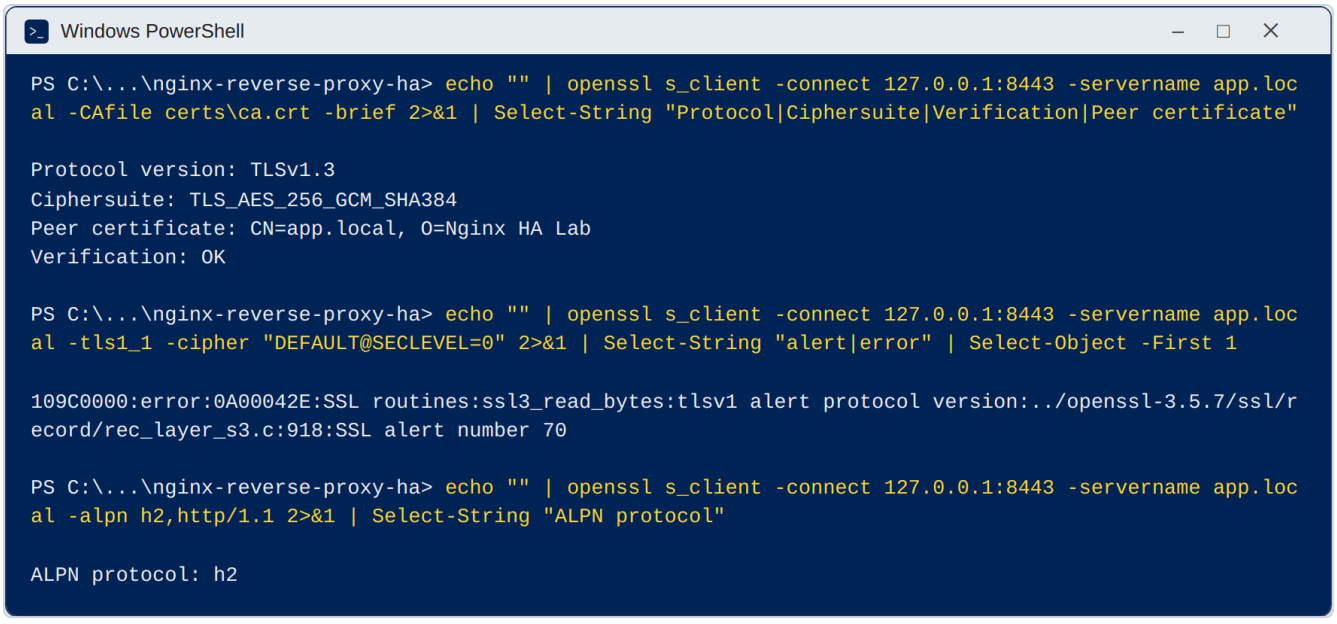


Figure 13: TLS 1.3 with a verified certificate; TLS 1.1 refused with a protocol-version alert; HTTP/2 negotiated

6.2 Trust the lab CA in your browser (optional)

PowerShell

```
Import-Certificate -FilePath certs\ca.crt -CertStoreLocation Cert:\CurrentUser\Root
```

After importing, **https://localhost:8443** opens without a certificate warning. Remove it later from *certmgr.msc → Trusted Root Certification Authorities* (“Nginx HA Lab Root CA”).

6.3 Rotate the certificate with zero downtime

PowerShell

```
.\lab.ps1 rotate-certs
```

Incident drill 8 (Step 9) runs this under live traffic and shows no failed requests.

Production: Let's Encrypt

bash (Ubuntu server)

```
sudo scripts/linux/06-ssl-setup.sh --staging        # lb1: test run
sudo scripts/linux/06-ssl-setup.sh                  # lb1: real certificate + renewal hook
sudo scripts/linux/06-ssl-setup.sh --receiver '<key>' # lb2: accept synced certificates
sudo scripts/linux/06-ssl-setup.sh --sync           # lb1: push the certificate to lb2
```

9. Step 7: High availability and load balancing

Nginx uses **least_conn**, which sends each request to the backend with the fewest active connections. If a backend fails (connection error, timeout, 502/503/504) Nginx retries the request on another one. After 3 failures in 10 seconds the backend is taken out of rotation for 10 seconds and then retried.

7.1 See the load being shared

PowerShell

```
.\lab.ps1 load --rps 10 --duration 8
```

Windows PowerShell

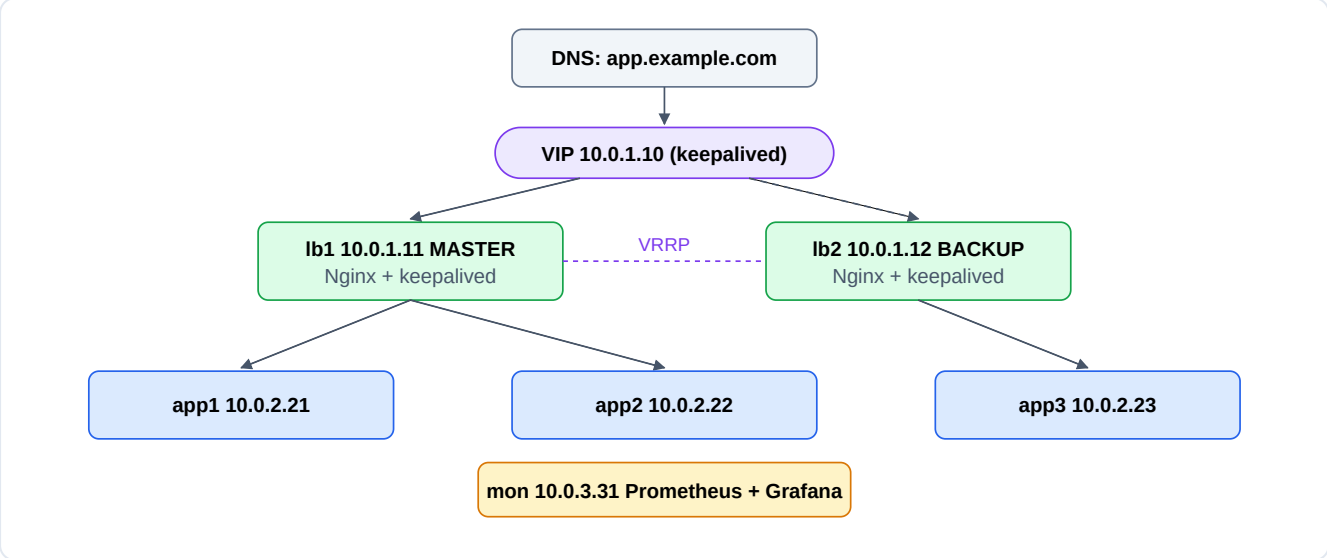
```
PS C:\...\nginx-reverse-proxy-ha> .\lab.ps1 load --rps 10 --duration 8
t=  0s  200:10                  backend1:3 backend2:3 backend3:4
t=  1s  200:10                  backend1:4 backend2:4 backend3:2
t=  2s  200:10                  backend1:3 backend2:3 backend3:4
… 3 per-second lines omitted …
t=  6s  200:10                  backend1:4 backend2:4 backend3:2
t=  7s  200:10                  backend1:3 backend2:3 backend3:4

==== summary ====
requests: 80   success(200): 80   success rate: 100.00%
status codes: {'200': 80}
served by: {'backend3': 26, 'backend1': 27, 'backend2': 27}
latency p95: 14 ms   max: 61 ms
```

Figure 14: 80 requests at 10 per second: 100% success, evenly spread over backend1, backend2 and backend3

7.2 Failover

Failover is demonstrated by incident drills 1-4 and 6 in Step 9. In production, a second Nginx server and keepalived provide a floating IP (VIP) that moves to the standby proxy within about 4 seconds if the active one fails:



bash (Ubuntu server)

```
sudo scripts/linux/07-ha-keepalived.sh    # run on lb1 first, then lb2
```

10. Step 8: Monitoring and Grafana dashboard

Prometheus collects metrics every 5 seconds from Nginx (via nginx-exporter), from each backend's `/metrics` endpoint, and from Blackbox probes that test HTTPS through the proxy (including certificate

validity) and each backend's `/health`. Ten alert rules cover outages, errors, latency and certificate expiry.

8.1 Open the monitoring tools

PowerShell

```
.\lab.ps1 urls
Start-Process "http://127.0.0.1:3300"
Select-String -Path .env -Pattern "GRAFANA_ADMIN_PASSWORD"
```

```
Windows PowerShell
PS C:\...\nginx-reverse-proxy-ha> .\lab.ps1 urls

  Application (HTTPS)   https://localhost:8443/         (trust certs/ca.crt to avoid the browser warnin
g)
  Application (HTTP)    http://localhost:8880/           -> redirects to HTTPS
  Grafana               http://127.0.0.1:3300/           user: admin  password: see GRAFANA_ADMIN_PASSW
ORD in .env
  Prometheus            http://127.0.0.1:9390/
  Prometheus alerts     http://127.0.0.1:9390/alerts
```

Figure 15: Endpoints of the lab

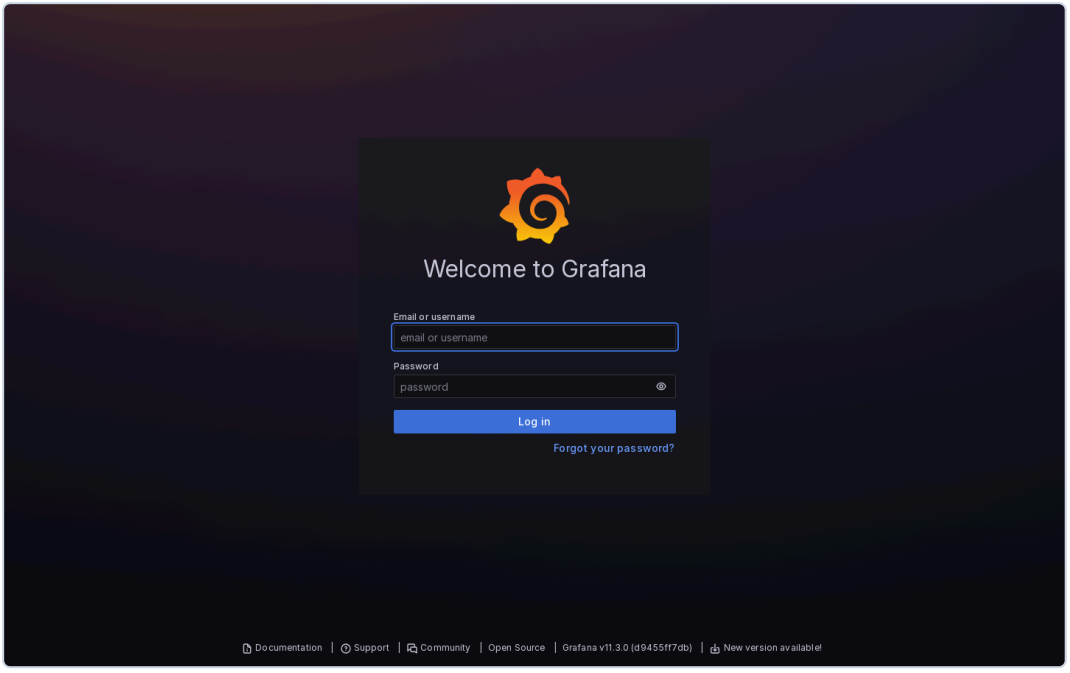


Figure 16: Grafana login (user admin, password from .env)

8.2 The dashboard

The dashboard *Nginx Reverse Proxy - HA Overview* is provisioned automatically and opens as the Grafana home page. The time range below covers the incident drills, which show up as dips and spikes.



Figure 17: Grafana dashboard: service health, requests per backend, backend up/down, p95 latency, responses by status, Nginx connections, probe duration and chaos-mode markers

Panel	What it tells you
Nginx / HTTPS endpoint	proxy up; end-to-end HTTPS probe passing
Healthy backends	how many of the 3 backends pass /health
Proxy requests / s, Backend 5xx ratio	traffic and error rate
TLS cert expires in	days left on the certificate
Requests per backend	load-balancing distribution; a failed node drops to zero
p95 latency per backend	slow nodes stand out immediately

8.3 Prometheus targets and alert rules

PowerShell

```
(Invoke-RestMethod "http://127.0.0.1:9390/api/v1/targets").data.activeTargets | Select-Object @{n="job";e={$_.labels.job}}, @{n="instance";e={$_.labels.instance}}, health | Format-Table -AutoSize
```

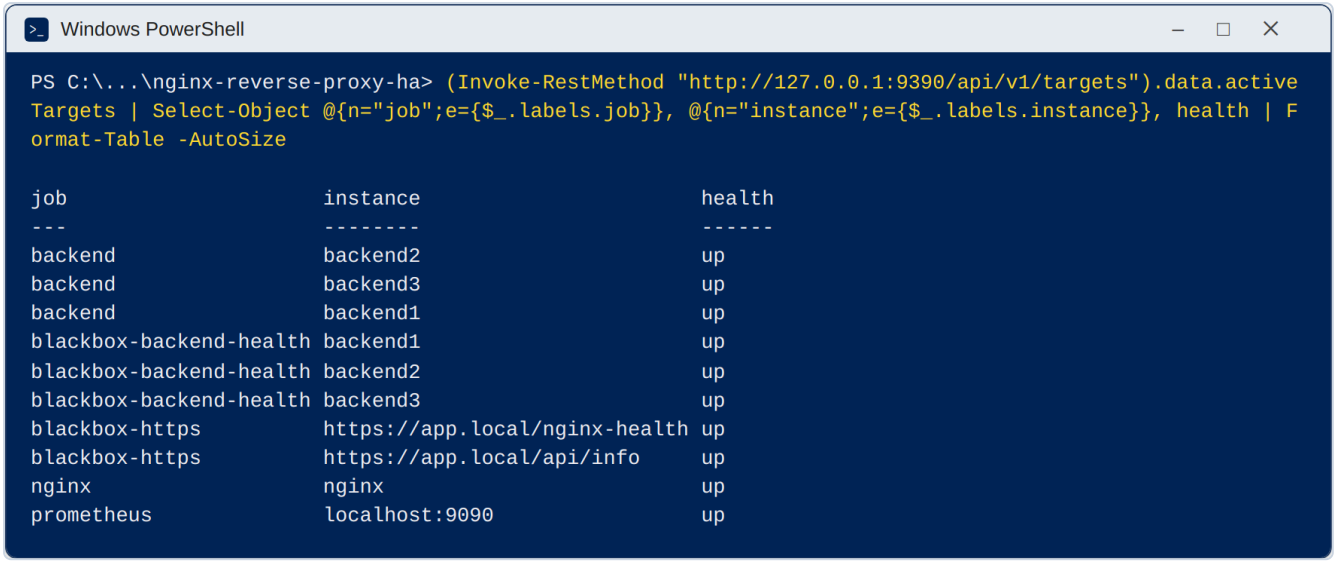


Figure 18: All 10 scrape targets are up

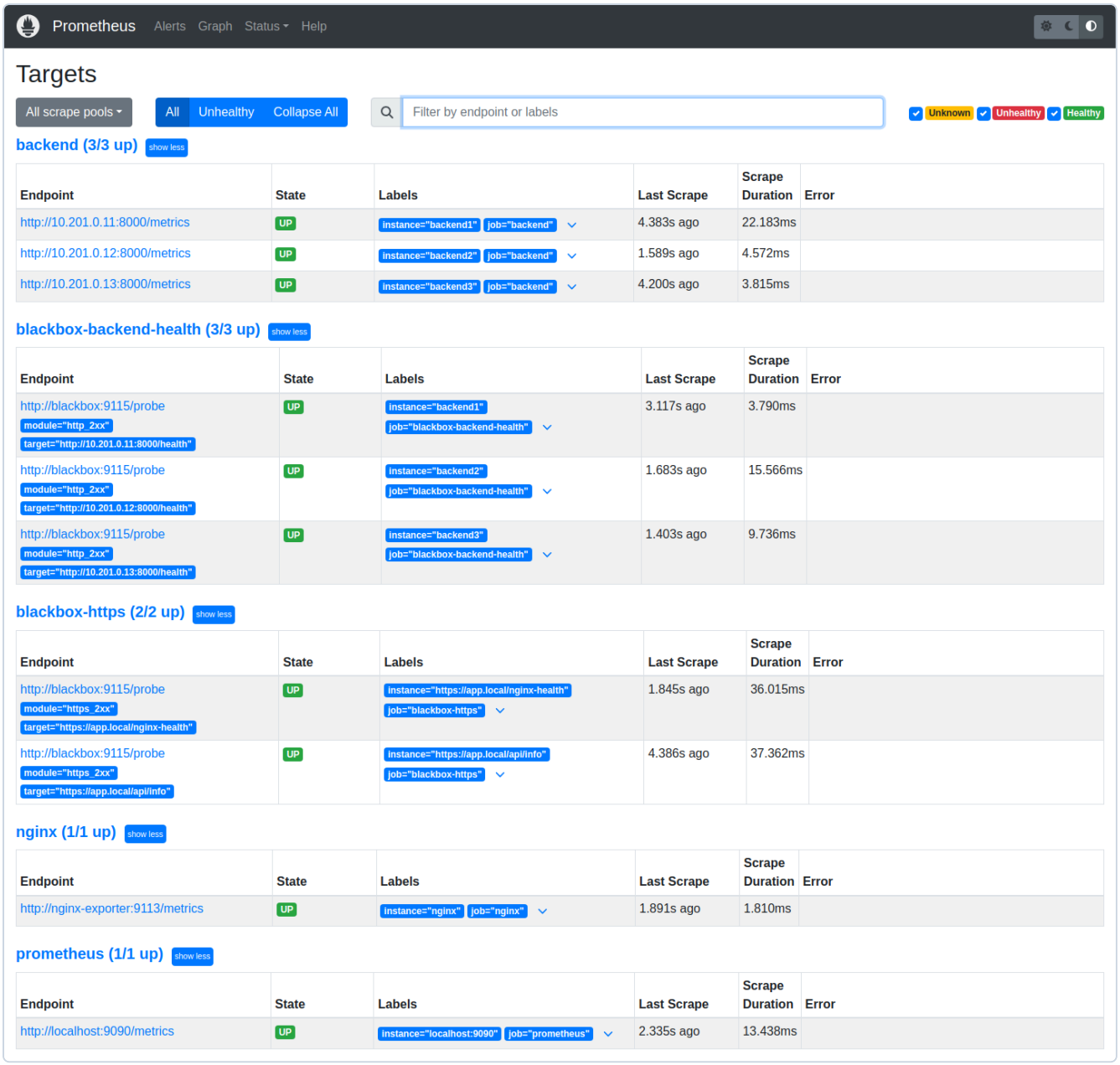


Figure 19: Prometheus targets page

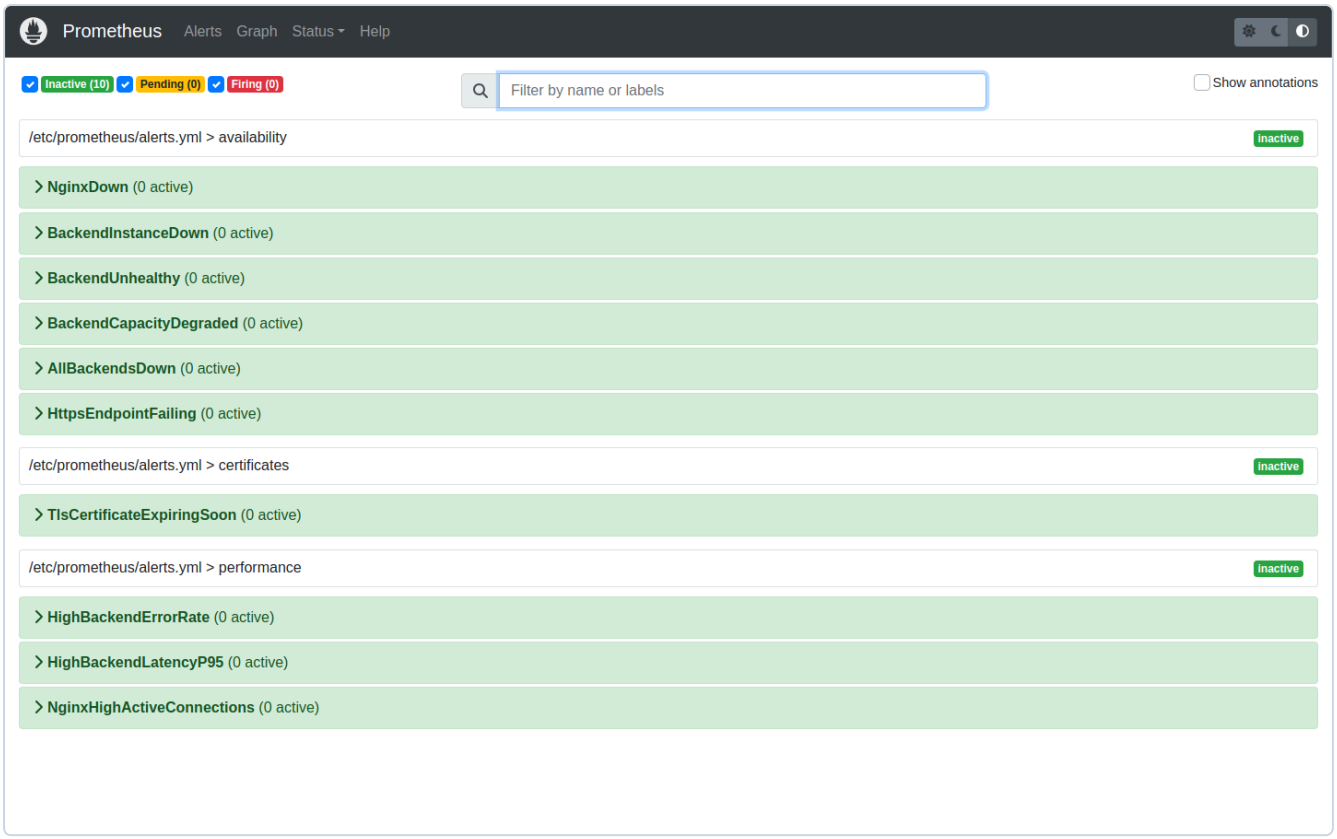


Figure 20: Prometheus alert rules, all inactive while the system is healthy

11. Step 9: Incident simulation and recovery

Each drill follows the same pattern: **inject** a failure while traffic is flowing, **detect** it through monitoring, **recover** using the runbook procedure (`docs/runbook.md`), and **verify** the outcome. A load generator sends requests during the drill and prints, per second, the status codes and which backend served them.

PowerShell

```
.\lab.ps1 incident list
```

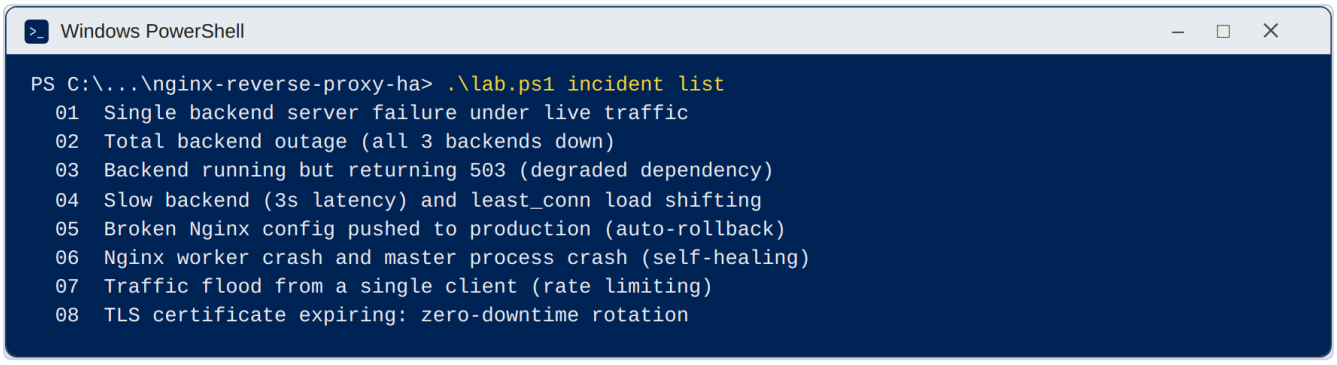


Figure 21: The eight incident drills

> **Tip:** Keep Grafana open while running drills to watch the dashboard react. Each drill writes a log to `reports/` .

Incident 1: Single backend failure under live traffic

Scenario: backend2 is stopped while 10 requests/second flow.

PowerShell

```
.\lab.ps1 incident 1
```

Windows PowerShell

```
PS C:\...\nginx-reverse-proxy-ha> .\lab.ps1 incident 1
======================================================================
 INCIDENT DRILL 01: Single backend server failure under live traffic
 started 2026-09-29 18:28:31
======================================================================

==> [18:28:32] Baseline: 10 req/s of user traffic for 75s

==> [18:28:45] INJECT: backend2 goes down (host lost / container stopped)

==> [18:28:46] DETECT: waiting for monitoring to alert
[ OK ] BackendInstanceDown alert FIRING (after 15s)
    up backend2 0
    up backend3 1
    up backend1 1

==> [18:29:11] IMPACT: users should be unaffected (Nginx fails over to backend1/backend3)
    GET /api/info -> 200
    GET /api/info -> 200
    GET /api/info -> 200
    GET /api/info -> 200
    GET /api/info -> 200

==> [18:29:13] RECOVER: restart backend2 (runbook INC-01)
[ OK ] backend2 container healthy (after 2s)

==> [18:29:22] VERIFY: backend2 serving traffic again, alert resolved
[ OK ] BackendInstanceDown alert cleared (after 0s)

==> Load generator results (per second: status codes, then which backend served)

    t=  0s  200:10                  backend1:3 backend2:4 backend3:3
    t=  1s  200:10                  backend1:3 backend2:3 backend3:4
    t=  2s  200:10                  backend1:4 backend2:3 backend3:3
    t=  3s  200:10                  backend1:3 backend2:4 backend3:3
    t=  4s  200:10                  backend1:4 backend2:3 backend3:3
    t=  5s  200:10                  backend1:3 backend2:4 backend3:3
… 66 per-second lines omitted …
    t= 72s  200:10                  backend1:3 backend2:4 backend3:3
    t= 73s  200:10                  backend1:3 backend2:3 backend3:4
    t= 74s  200:10                  backend1:3 backend2:4 backend3:3

    ==== summary ====
    requests: 750   success(200): 750   success rate: 100.00%
    status codes: {'200': 750}
    served by: {'backend2': 127, 'backend3': 312, 'backend1': 311}
    latency p95: 17 ms   max: 1269 ms

[INFO] Expected: 100% success; backend2 disappears from the per-second counts during the outage, then re
turns.

Drill finished 18:29:51. Log: reports/incident-01-20260929-182831.log
```

Figure 22: Incident 1: Single backend failure under live traffic (real run)

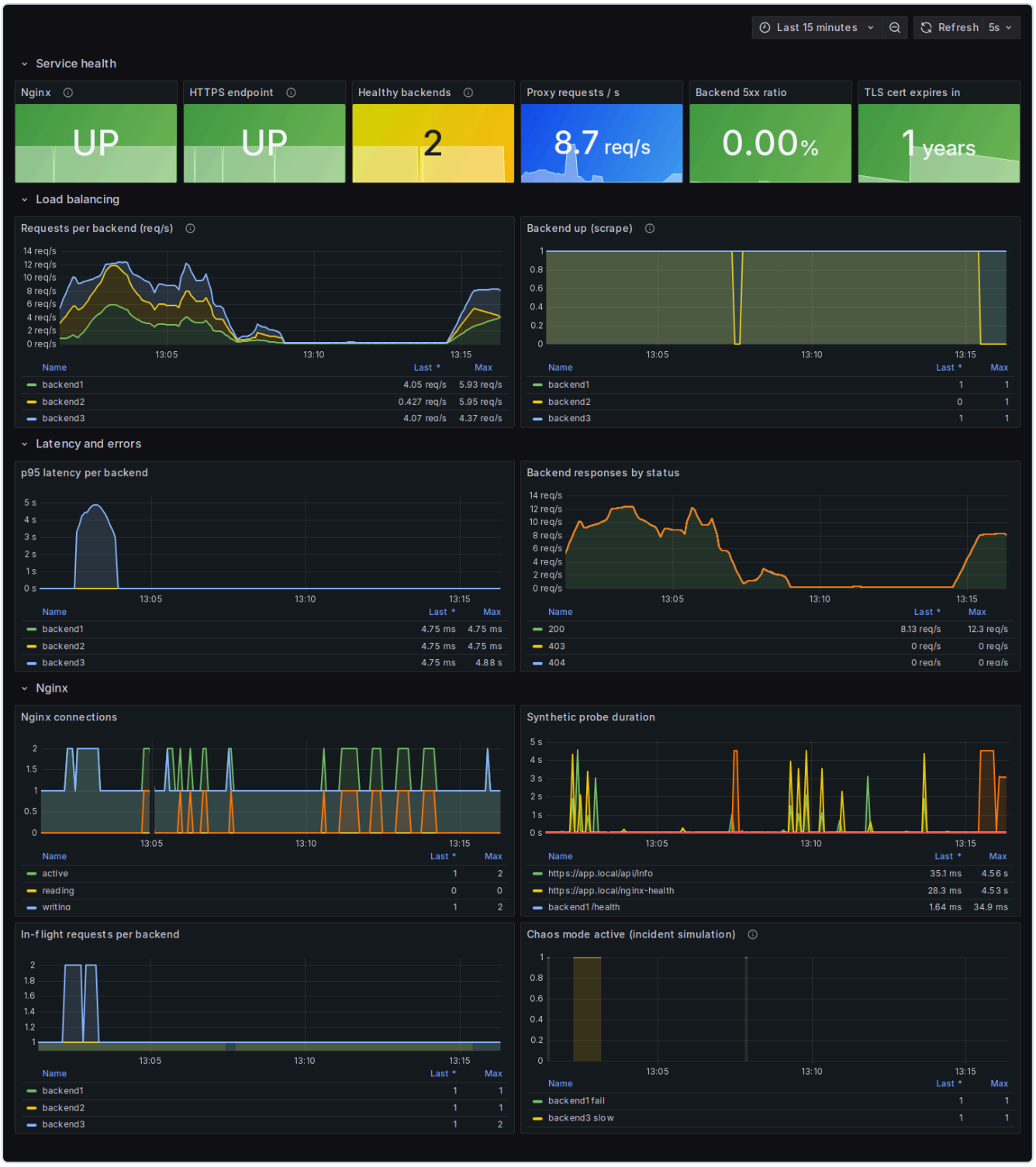


Figure 23: Grafana while backend2 is down: Healthy backends drops to 2, backend2 stops receiving traffic and shows as down

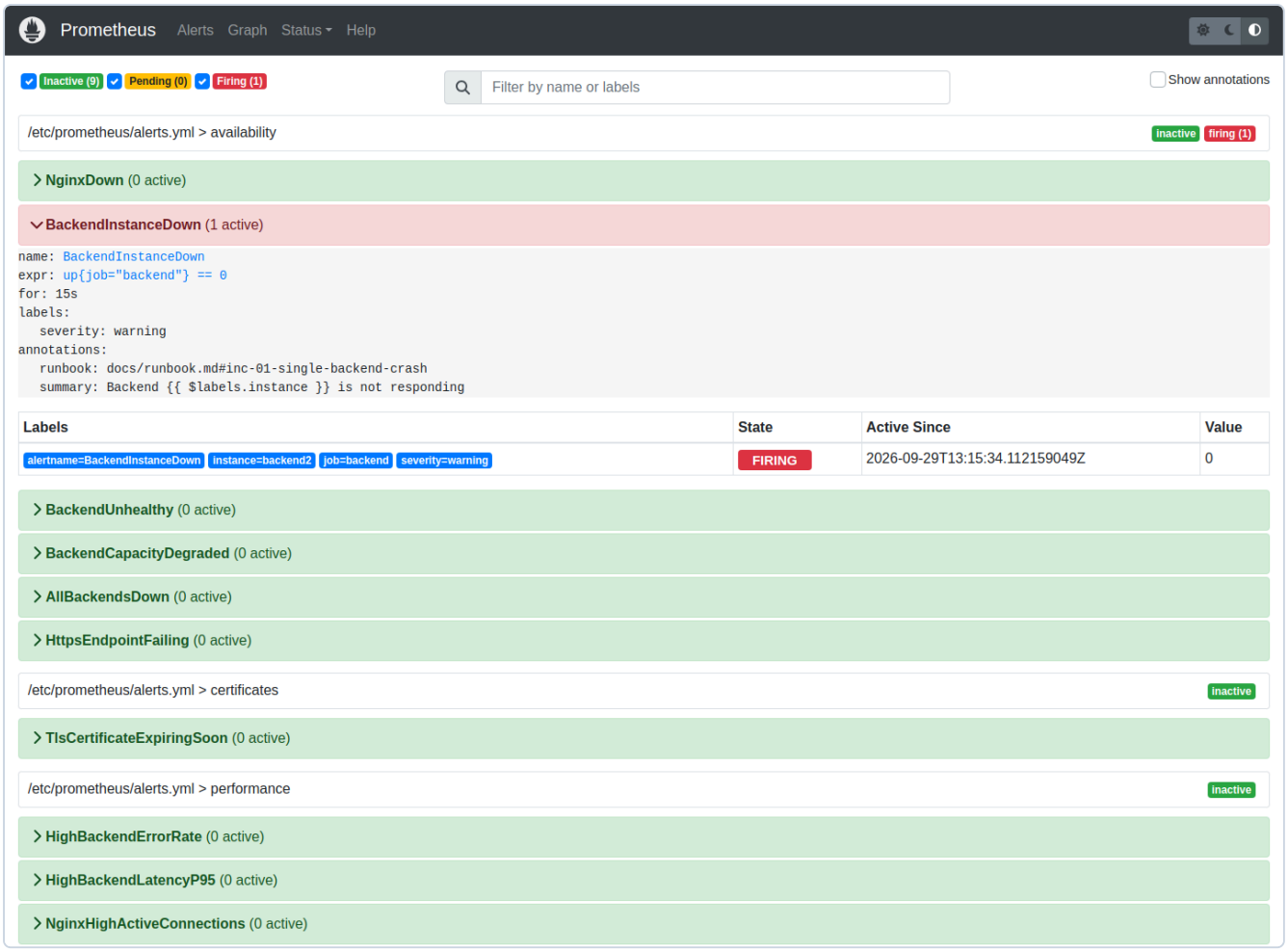


Figure 24: Prometheus showing BackendInstanceDown firing for backend2

> **Result:** Nginx fails over to backend1 and backend3; the BackendInstanceDown alert fires; backend2 is restarted and rejoins automatically. **No failed requests.**

Incident 2: Total backend outage

Scenario: All three backends are stopped.

PowerShell

```
.\lab.ps1 incident 2
```

```
Windows PowerShell
PS C:\...\nginx-reverse-proxy-ha> .\lab.ps1 incident 2
========================================================================
 INCIDENT DRILL 02: Total backend outage (all 3 backends down)
 started 2026-09-29 18:29:53
========================================================================

==> [18:29:54] Baseline
    GET /api/info -> 200

==> [18:29:55] INJECT: all backends stop (bad release / shared dependency failure)

==> [18:29:57] IMPACT: proxy stays up and serves a branded maintenance page, not a raw error
    GET /api/info     -> 504
    GET /nginx-health -> 200   (proxy itself is alive)
    error page body   -> <h1>We'll be right back</h1>

==> [18:30:04] DETECT
[ OK ] AllBackendsDown alert FIRING (after 8s)
[ OK ] HttpsEndpointFailing alert FIRING (after 3s)
    AllBackendsDown (firing)
    BackendCapacityDegraded (firing)
    BackendInstanceDown (firing)
    HttpsEndpointFailing (firing)

==> [18:30:22] RECOVER: start backends one by one; service resumes as soon as the first is healthy
[ OK ] service restored (first backend healthy) (after 2s)
    time to first successful response: 6s

==> [18:30:29] VERIFY
          5 backend1
          4 backend2
          3 backend3
[ OK ] AllBackendsDown alert cleared (after 0s)

Drill finished 18:30:46. Log: reports/incident-02-20260929-182953.log
```

Figure 25: Incident 2: Total backend outage (real run)

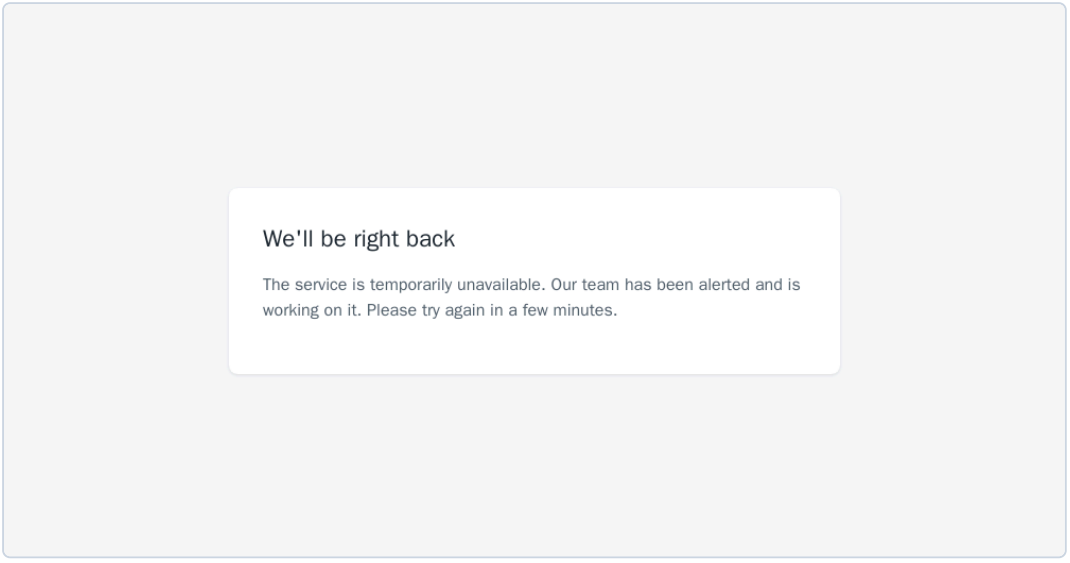


Figure 26: What users see when every backend is down: a branded page, not a raw 502

> **Result:** Nginx stays up and serves a branded maintenance page instead of a raw error; AllBackendsDown and HttpsEndpointFailing fire; service returns as soon as the first backend is started.

Incident 3: Degraded backend (returns 503)

Scenario: backend1 keeps running but fails every request, like a lost database connection.

PowerShell

```
.\lab.ps1 incident 3
```

Windows PowerShell

```
PS C:\...\nginx-reverse-proxy-ha> .\lab.ps1 incident 3
======================================================================
 INCIDENT DRILL 03: Backend running but returning 503 (degraded dependency)
 started 2026-09-29 18:30:48
======================================================================

==> [18:30:49] Baseline: 10 req/s for 60s

==> [18:31:00] INJECT: backend1 starts failing every request with 503 (e.g. lost its DB connection)
    {"delay_ms":2000,"instance":"backend1","mode":"fail"}

==> [18:31:03] DETECT: process is up (scrape OK) but health check fails
[ OK ] BackendUnhealthy alert FIRING (after 15s)
    health backend1 0
    health backend2 1
    health backend3 1

==> [18:31:26] IMPACT: Nginx retries 503s on the next backend (proxy_next_upstream), so users see 200
    GET /api/info -> 200
    GET /api/info -> 200
    GET /api/info -> 200
    GET /api/info -> 200
    GET /api/info -> 200

==> [18:31:28] RECOVER: fix the dependency / restart the instance
    {"delay_ms":2000,"instance":"backend1","mode":"ok"}

==> [18:31:30] VERIFY
[ OK ] BackendUnhealthy alert cleared (after 3s)

    t=  0s  200:10                  backend1:3 backend2:3 backend3:4
    t=  1s  200:10                  backend1:4 backend2:3 backend3:3
    t=  2s  200:10                  backend1:3 backend2:4 backend3:3
… 55 per-second lines omitted …
    t= 58s  200:10                  backend1:4 backend2:3 backend3:3
    t= 59s  200:10                  backend1:3 backend2:3 backend3:4

    ==== summary ====
    requests: 600   success(200): 600   success rate: 100.00%
    status codes: {'200': 600}
    served by: {'backend3': 255, 'backend1': 86, 'backend2': 259}
    latency p95: 17 ms   max: 85 ms
[INFO] Expected: 100% success; backend1 serves nothing during the fault and resumes after recovery.

Drill finished 18:31:53. Log: reports/incident-03-20260929-183048.log
```

Figure 27: Incident 3: Degraded backend (returns 503) (real run)

> **Result:** Nginx retries the failed requests on healthy backends; BackendUnhealthy fires. **No failed requests.**

Incident 4: Slow backend

Scenario: backend3 starts answering in 3 seconds.

PowerShell

```
.\lab.ps1 incident 4
```

Windows PowerShell

```
PS C:\...\nginx-reverse-proxy-ha> .\lab.ps1 incident 4
======================================================================
 INCIDENT DRILL 04: Slow backend (3s latency) and least_conn load shifting
 started 2026-09-29 18:31:55
======================================================================

==> [18:31:56] Baseline: 12 req/s for 100s

==> [18:32:09] INJECT: backend3 responses slow down to 3000 ms (GC pauses / noisy neighbour)
    {"delay_ms":3000,"instance":"backend3","mode":"slow"}

==> [18:32:11] OBSERVE: least_conn sends fewer requests to the backend that holds connections open
    backend1  5.8 req/s
    backend3  0.5 req/s
    backend2  5.7 req/s

==> [18:32:37] DETECT
[ OK ] HighBackendLatencyP95 alert FIRING (after 19s)
    p95 backend3  4.875s
    p95 backend2  0.005s
    p95 backend1  0.005s

==> [18:33:11] RECOVER: remove the slow instance / fix it
    {"delay_ms":2000,"instance":"backend3","mode":"ok"}

    t=  0s  200:12                  backend1:4 backend2:4 backend3:4
    t=  1s  200:12                  backend1:4 backend2:4 backend3:4
    t=  2s  200:12                  backend1:4 backend2:4 backend3:4
… 95 per-second lines omitted …
    t= 98s  200:12                  backend1:4 backend2:4 backend3:4
    t= 99s  200:12                  backend1:4 backend2:4 backend3:4

    ==== summary ====
    requests: 1200   success(200): 1200   success rate: 100.00%
    status codes: {'200': 1200}
    served by: {'backend3': 161, 'backend2': 522, 'backend1': 517}
    latency p95: 17 ms   max: 3018 ms
[INFO] Expected: 100% success. backend3's share drops while slow (least_conn), max latency ~3s,
[INFO] no timeouts (proxy_read_timeout 30s). If slowness exceeded 30s Nginx would retry elsewhere.

Drill finished 18:33:40. Log: reports/incident-04-20260929-183155.log
```

Figure 28: Incident 4: Slow backend (real run)

> **Result:** least_conn automatically sends it far fewer requests; HighBackendLatencyP95 fires. **No failed requests.**

Incident 5: Broken configuration deployed

Scenario: A typo (`proxy_pas`) is deployed through the safe-reload pipeline.

PowerShell

```
.\lab.ps1 incident 5
```

```
Windows PowerShell
PS C:\...\nginx-reverse-proxy-ha> .\lab.ps1 incident 5
======================================================================
 INCIDENT DRILL 05: Broken Nginx config pushed to production (auto-rollback)
 started 2026-09-29 18:33:42
======================================================================

==> [18:33:44] Baseline: 10 req/s for 40s

==> [18:33:52] INJECT: an engineer deploys a config with a typo (proxy_pass -> proxy_pas)
    90:        proxy_pas http://backend_pool;

==> [18:33:52] DEPLOY via the safe pipeline (./lab.sh reload): validate -> reload or roll back

==> Validating and reloading Nginx configuration
2026/09/29 13:03:54 [emerg] 1078#1078: unknown directive "proxy_pas" in /etc/nginx/generated/app.conf:90
nginx: [emerg] unknown directive "proxy_pas" in /etc/nginx/generated/app.conf:90
nginx: configuration file /etc/nginx/nginx.conf test failed
!! config test failed, restoring previous config
previous config restored and valid
[FAIL] reload aborted; the running config was left in place
[ OK ] bad config rejected by 'nginx -t'; previous config restored; running workers never touched it

==> [18:33:54] IMPACT check: traffic still flowing on the last good config
    GET /api/info -> 200
    GET /api/info -> 200
    GET /api/info -> 200
    nginx: configuration file /etc/nginx/nginx.conf test is successful

==> [18:33:56] RECOVER: revert the change in source control and redeploy

==> Validating and reloading Nginx configuration
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
2026/09/29 13:03:58 [notice] 1108#1108: signal process started
[ OK ] reloaded without dropping connections

==> [18:33:59] VERIFY

    t=  0s  200:10                  backend1:3 backend2:4 backend3:3
    t=  1s  200:10                  backend1:3 backend2:3 backend3:4
    t=  2s  200:10                  backend1:4 backend2:3 backend3:3
… 35 per-second lines omitted …
    t= 38s  200:10                  backend1:3 backend2:3 backend3:4
    t= 39s  200:10                  backend1:4 backend2:3 backend3:3

    ==== summary ====
    requests: 400   success(200): 400   success rate: 100.00%
    status codes: {'200': 400}
    served by: {'backend2': 128, 'backend3': 137, 'backend1': 135}
    latency p95: 10 ms   max: 61 ms
[INFO] Expected: 100% success. Without the validation gate, the next Nginx restart would have
[INFO] failed to start: a full outage caused by a one-character typo.

Drill finished 18:34:28. Log: reports/incident-05-20260929-183342.log
```

Figure 29: Incident 5: Broken configuration deployed (real run)

> **Result:** `nginx -t` rejects it, the previous config is restored automatically, traffic is never affected.

Incident 6: Nginx crash

Scenario: A worker process, then the Nginx master process, is killed with SIGKILL.

PowerShell

```
.\lab.ps1 incident 6
```

Windows PowerShell

```
PS C:\...\nginx-reverse-proxy-ha> .\lab.ps1 incident 6
======================================================================
 INCIDENT DRILL 06: Nginx worker crash and master process crash (self-healing)
 started 2026-09-29 18:34:30
======================================================================

==> [18:34:31] Baseline: 10 req/s for 45s

==> [18:34:38] INJECT A: a worker process is SIGKILLed (segfault / module bug)
    workers before: 1109 1110 1111 1112 1113 1114 1115 1116 1141
    workers after:  1110 1111 1112 1113 1114 1115 1116 1160 1167
[ OK ] master respawned the worker immediately

==> [18:34:51] INJECT B: the master process is SIGKILLed (OOM killer)

==> [18:34:56] RECOVER: Docker restart policy (unless-stopped) brings it back; systemd Restart=always do
es the same on VMs
[ OK ] HTTPS serving again (after 0s)
    outage window: 18:34:56 -> 18:34:58  (~5s)
    container restart count: 3 -> 4

==> [18:35:00] VERIFY

    t=  0s  200:10                  backend1:3 backend2:3 backend3:4
    t=  1s  200:10                  backend1:3 backend2:4 backend3:3
    t=  2s  200:10                  backend1:3 backend2:3 backend3:4
    t=  3s  200:10                  backend1:4 backend2:3 backend3:3
    t=  4s  200:10                  backend1:3 backend2:4 backend3:3
    t=  5s  200:10                  backend1:3 backend2:3 backend3:4
… 34 per-second lines omitted …
    t= 22s  200:1 ERR:3             backend3:1
    t= 23s  200:4 ERR:4             backend1:2 backend2:1 backend3:1
    t= 42s  200:10                  backend1:4 backend2:3 backend3:3
    t= 43s  200:10                  backend1:3 backend2:4 backend3:3
    t= 44s  200:10                  backend1:3 backend2:3 backend3:4

    ==== summary ====
    requests: 450   success(200): 442   success rate: 98.22%
    status codes: {'200': 442, 'ERR': 8}
    served by: {'backend3': 153, 'backend1': 144, 'backend2': 145}
    latency p95: 17 ms   max: 2510 ms
[INFO] Expected: no errors for the worker crash; a short burst of connection errors (ERR) for the
[INFO] master crash, then full recovery. In production the keepalived VIP (scripts/linux/07) moves to
[INFO] the standby proxy, so users don't see even that gap.

Drill finished 18:35:20. Log: reports/incident-06-20260929-183429.log
```

Figure 30: Incident 6: Nginx crash (real run)

> **Result:** The worker is respawned instantly with no errors. The master crash causes a sub-second gap until Docker restarts Nginx; in production the keepalived VIP moves to the standby proxy instead.

Incident 7: Traffic flood from one client

Scenario: One client sends 120 requests/second.

PowerShell

```
.\lab.ps1 incident 7
```

Windows PowerShell

```
PS C:\...\nginx-reverse-proxy-ha> .\lab.ps1 incident 7
======================================================================
 INCIDENT DRILL 07: Traffic flood from a single client (rate limiting)
 started 2026-09-29 18:35:21
======================================================================

==> [18:35:23] INJECT: one client sends 120 req/s for 15s (scraper / credential stuffing / L7 flood)

==> [18:35:30] IMPACT on OTHER users (different source IP): should be unaffected
    legitimate user, 10 requests:  10 200,

==> [18:35:38] Nginx error log shows the limiter working
    limited requests logged in last 10s: 961

==> Attacker's results: excess requests get 429 at the edge and never reach the backends
    ==== summary ====
    requests: 1800   success(200): 340   success rate: 18.89%
    status codes: {'200': 340, '429': 1460}
    served by: {'backend3': 113, 'backend2': 113, 'backend1': 114}
    latency p95: 11 ms   max: 38 ms
[INFO] Policy: 20 req/s per IP, burst 40, max 50 concurrent connections per IP (nginx/nginx.conf).
[INFO] For a distributed flood, add upstream protection (CDN/WAF) and fail2ban on the 429 log lines.

Drill finished 18:35:42. Log: reports/incident-07-20260929-183521.log
```

Figure 31: Incident 7: Traffic flood from one client (real run)

> **Result:** Excess requests get HTTP 429 at the edge; a normal user at the same time is unaffected.

Incident 8: Certificate rotation

Scenario: A new certificate is issued and loaded while traffic flows.

PowerShell

```
.\lab.ps1 incident 8
```

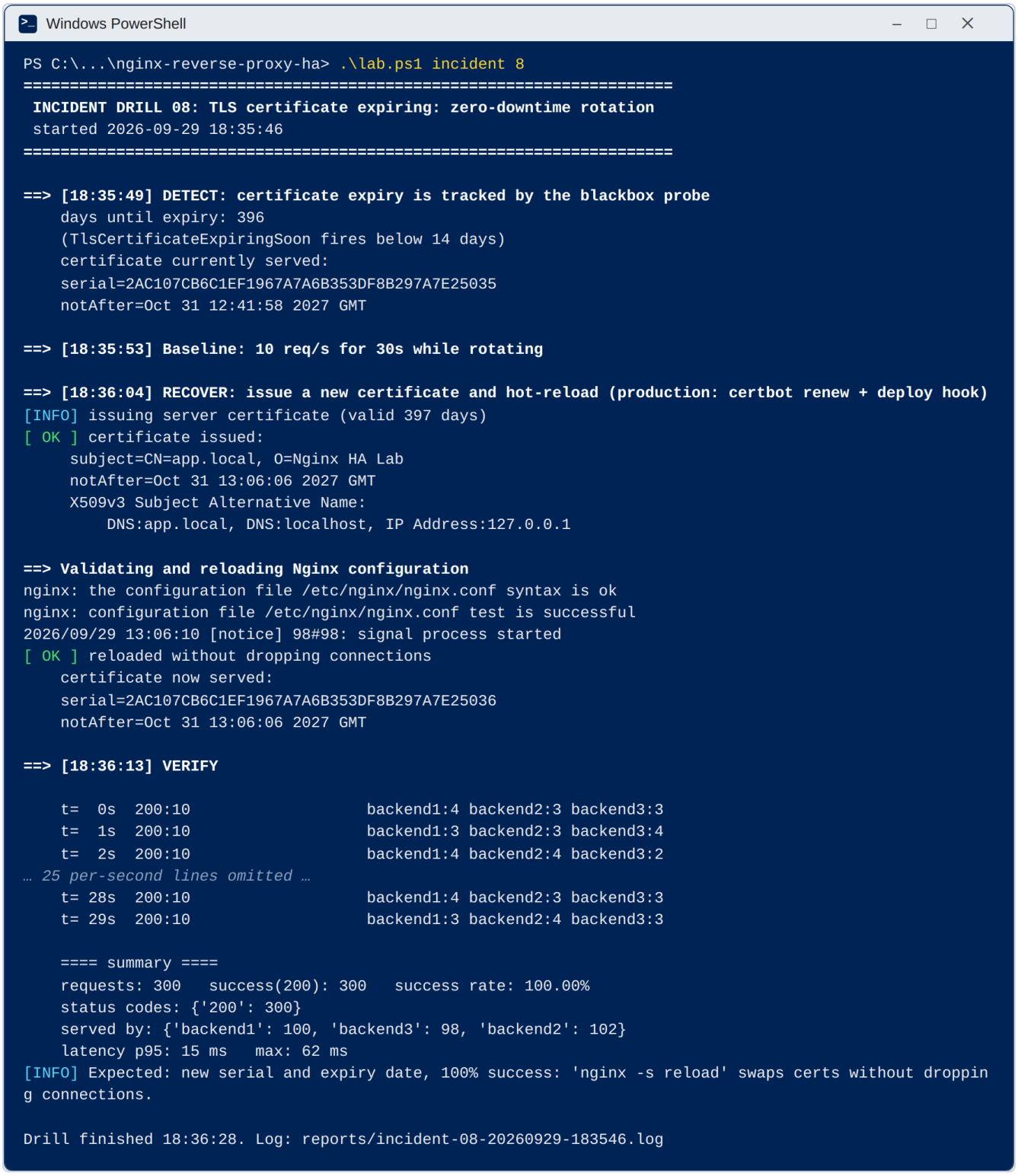


Figure 32: Incident 8: Certificate rotation (real run)

> **Result:** New serial number served, hot reload, **no failed requests.**

12. Step 10: Testing and documentation

10.1 Automated acceptance tests

46 checks covering the proxy, TLS, security headers, backend isolation, load balancing, failover, rate limiting, monitoring and configuration safety. A Markdown report is written to `reports/`.

PowerShell

```
.\lab.ps1 test
```

Windows PowerShell

```
PS C:\...\nginx-reverse-proxy-ha> .\lab.ps1 test

==> Pre-flight
backend1 backend2 backend3 blackbox grafana nginx nginx-exporter prometheus

==> Reverse proxy
[ OK ] T01 HTTPS request is proxied to a backend  [status=200]
[ OK ] T02 X-Forwarded-Proto/Request-ID reach the backend  ["proto":"https"]
[ OK ] T03 Proxy liveness endpoint  [status=200]
[ OK ] T03b Error-page stylesheet served by Nginx (CSP blocks inline styles)  [200 text/css]
[ OK ] T04 Unknown route returns backend 404 (not 5xx)  [status=404]

==> SSL / HTTPS
[ OK ] T05 HTTP redirects permanently to HTTPS (path + query kept)  [301 https://127.0.0.1:8443/api/inf
o?x=1]
[ OK ] T06 Certificate chain validates against the CA  [Verify return code: 0 (ok)]
[ OK ] T07 TLSv1.3 accepted  [New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384]
[ OK ] T08 TLSv1.2 accepted  [New, TLSv1.2, Cipher is ECDHE-RSA-AES256-GCM-SHA384]
[ OK ] T09 TLSv1.1 rejected  [tls1.1 rejected]
[ OK ] T10 TLSv1.0 rejected  [tls1.0 rejected]
[ OK ] T11 HTTP/2 negotiated via ALPN  [ALPN protocol: h2]
[ OK ] T12 TLS handshake rejected for unknown hostname (SNI)  [sni=evil.example rejected]

==> Security headers and hardening
[ OK ] T13 Header present: Strict-Transport-Security  [Strict-Transport-Security: max-age=31536000; incl
udeSubDomai]
[ OK ] T13 Header present: X-Frame-Options  [X-Frame-Options: DENY]
[ OK ] T13 Header present: X-Content-Type-Options  [X-Content-Type-Options: nosniff]
[ OK ] T13 Header present: Content-Security-Policy  [Content-Security-Policy: default-src 'self'; frame-
ancestors]
[ OK ] T13 Header present: Referrer-Policy  [Referrer-Policy: strict-origin-when-cross-origin]
[ OK ] T13 Header present: X-Request-ID  [X-Request-ID: b68f40cf23258448b61aa0ad37d8aeab]
[ OK ] T14 Server header hides Nginx version  [Server: nginx]
[ OK ] T15 TRACE method blocked  [status=405]
[ OK ] T16 /metrics not exposed publicly  [status=404]
[ OK ] T17 /admin/* not exposed publicly  [status=404]
[ OK ] T18 Dotfiles (.env/.git) blocked  [status=404]

==> Backend security
[ OK ] T19 Backend port 8000 not reachable from host  [status=000]
[ OK ] T20 Nginx status port not published to host  [status=404]
[ OK ] T21 Backends attached only to the internal network  [nginx-ha-lab_backend_net ]
[ OK ] T22 Backend network is internal (no egress)  [internal=true]
[ OK ] T23 Direct call without proxy token is rejected  [status=403]
[ OK ] T24 Backend has no internet egress  [egress=blocked]
[ OK ] T25 Backend runs as non-root  [uid=10001]
[ OK ] T26 Backend root filesystem is read-only  [read-only]

==> Load balancing and high availability
[ OK ] T27 Traffic distributed across all 3 backends  [ 9 backend1, 11 backend2, 10 backend3,]
[ OK ] T28 Zero failed requests with 1 backend stopped  [ 20 200,]
[ OK ] T28b Upstream traffic stays on the private subnet while a backend is down  [non-private upstream
s: none]
[ OK ] T29 Recovered backend automatically rejoins the pool  [seen=backend2 after 8s]
[ OK ] T30 503 from a degraded backend is retried on a healthy one  [ 20 200,]
[ OK ] T31 Rate limiting returns 429 above 20 r/s + burst 40 (150 r/s for 2s)  [200=79 429=221]

==> Monitoring
[ OK ] T32 All Prometheus targets healthy  [ 10 "health":"up",]
[ OK ] T33 Alert rules loaded  [rules=10]
[ OK ] T34 Grafana healthy  ["database": "ok"]
[ OK ] T35 Grafana dashboard provisioned  ["title":"Nginx Reverse Proxy - HA Overview"]
[ OK ] T36 Grafana rejects unauthenticated API access  [status=401]
[ OK ] T37 Certificate expiry is monitored  [days_left=396]

==> Configuration safety
```

```
[ OK ] T38 Safe reload of valid config succeeds  [reload=ok]
[ OK ] T39 nginx -t passes on the live config  [nginx: configuration file /etc/nginx/nginx.conf test is
successful]

[ OK ] 46/46 tests passed   report: reports/test-report-20260929-184913.md
```

Figure 33: All 46 acceptance tests pass

10.2 Safe configuration reload

Every config change goes through a gate: back up, render, `nginx -t`, then reload, or roll back automatically.

PowerShell

```
.\lab.ps1 reload
```

Windows PowerShell

```
PS C:\...\nginx-reverse-proxy-ha> .\lab.ps1 reload

==> Validating and reloading Nginx configuration
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
2026/09/29 12:58:04 [notice] 835#835: signal process started
[ OK ] reloaded without dropping connections
```

Figure 34: Validated, zero-downtime reload

10.3 Validate the production configurations

Renders the production templates with the example inventory and checks them with the real tools: `nginx -t`, `keepalived -t` and `promtool`.

PowerShell

```
& "C:\Program Files\Git\bin\bash.exe" tests/validate-production-configs.sh
```

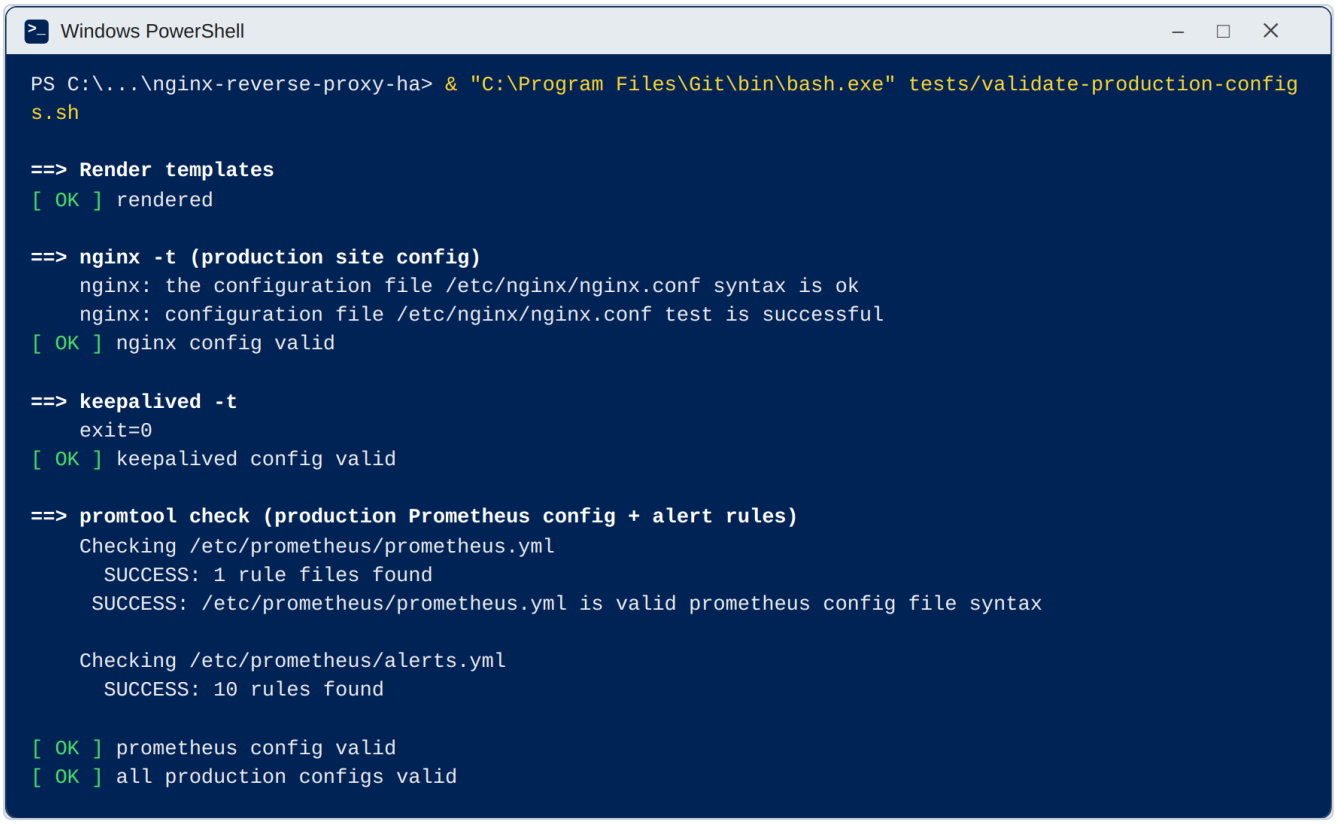


Figure 35: Production Nginx, keepalived and Prometheus configurations are valid

10.4 Documentation delivered

Document	Contents
README.md	overview, quick start, phase map
docs/architecture.md	topology, request flow, security layers, HA design, design decisions
docs/runbook.md	detect / diagnose / recover procedure for all 8 incidents
docs/production-deployment.md	deploying phases 1-10 on Ubuntu servers
docs/test-report.md	test results, drill results, findings
this PDF	step-by-step guide with commands and screenshots

13. Production deployment on Ubuntu servers

Run these on real servers (Ubuntu 22.04/24.04). Copy the project folder, including the edited `inventory.env`, to `/opt/nginx-ha` on every host first.

Host	IP (example)	Role
lb1, lb2	10.0.1.11, 10.0.1.12 (VIP 10.0.1.10)	Nginx + keepalived
app1, app2, app3	10.0.2.21-23	backend
mon	10.0.3.31	Prometheus + Grafana

Phase 1: Infrastructure preparation (once)

bash (Ubuntu server)

```
cp scripts/linux/inventory.env.example scripts/linux/inventory.env
nano scripts/linux/inventory.env        # domain, IPs, admin CIDR, e-mail
bash scripts/linux/01-infra-prep.sh      # validates, generates secrets
```

Phase 2: Server configuration

bash (Ubuntu server)

```
sudo scripts/linux/02-server-config.sh lb          # lb1, lb2
sudo scripts/linux/02-server-config.sh backend     # app1-3
sudo scripts/linux/02-server-config.sh monitoring  # mon
```

Phase 3: Backend deployment (app1-3)

bash (Ubuntu server)

```
sudo scripts/linux/03-deploy-backend.sh
```

Phase 4: Nginx (lb1, lb2)

bash (Ubuntu server)

```
sudo scripts/linux/04-install-nginx.sh
```

Phase 5: Firewall

bash (Ubuntu server)

```
sudo scripts/linux/05-firewall.sh lb
sudo scripts/linux/05-firewall.sh backend
sudo scripts/linux/05-firewall.sh monitoring
```

Phase 6: SSL (Let's Encrypt)

bash (Ubuntu server)

```
sudo scripts/linux/06-ssl-setup.sh              # lb1
sudo scripts/linux/06-ssl-setup.sh --receiver '<key printed on lb1>'   # lb2
sudo scripts/linux/06-ssl-setup.sh --sync       # lb1
```

Phase 7: High availability

bash (Ubuntu server)

```
sudo scripts/linux/07-ha-keepalived.sh   # lb1, then lb2
```

Phase 8: Monitoring

bash (Ubuntu server)

```
sudo scripts/linux/08-monitoring.sh lb          # lb1, lb2
sudo scripts/linux/08-monitoring.sh monitoring   # mon
```

Phase 9: Game day (maintenance window)

bash (Ubuntu server)

```
ssh app2 sudo systemctl stop backend    # backend failover
ssh lb1 sudo systemctl stop nginx       # VIP moves to lb2 in ~4 s
journalctl -t keepalived-notify -f      # watch the failover
```

Phase 10: Verify

bash (Ubuntu server)

```
curl -I http://app.example.com                      # 301 to https
curl -sI https://app.example.com                    # security headers
curl -m 3 http://10.0.2.21:8000/health              # from outside: must time out
```

14. Command cheat sheet and troubleshooting

Task	Command (PowerShell, in the project folder)
Allow scripts (each new window)	`Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force`
First-time setup	`.\lab.ps1 setup`
Start / stop	`.\lab.ps1 up` / `.\lab.ps1 down`
Health overview	`.\lab.ps1 status`
Send traffic	`.\lab.ps1 load --rps 10 --duration 30`
Run an incident drill	`.\lab.ps1 incident 1` … `8`
Run all tests	`.\lab.ps1 test`
Safe config reload	`.\lab.ps1 reload`
Rotate certificate	`.\lab.ps1 rotate-certs`
Follow logs	`.\lab.ps1 logs nginx`
Stop a backend by hand	`docker compose stop backend2` / `start backend2`
Delete everything	`.\lab.ps1 destroy`

Troubleshooting

Problem	Cause	Fix
“running scripts is disabled on this system”	execution policy	run the Set-ExecutionPolicy command above
“Git Bash not found”	Git for Windows missing	install from git-scm.com
docker: cannot connect / pipe not found	Docker Desktop not running	start Docker Desktop, wait for “Engine running”
port is already allocated	another program uses 8880/8443/3300/9390	change the ports in `.env`, then `.\lab.ps1 up`
browser certificate warning	lab CA not trusted	import `certs\ca.crt` (Step 6.2) or click through
`curl` prints HTML / errors in PowerShell	`curl` is an alias for Invoke-WebRequest	use `curl.exe`
a test fails right after a drill	an alert or ejected backend is still recovering	wait 30 s and run `.\lab.ps1 test` again

15. Results summary

Area	Result
Acceptance tests	**46 / 46 passed**
Backend crash / degraded / slow	**0 failed requests** (750, 600 and 1200 requests)
All backends down	branded page, proxy stayed up, alerts fired, recovery in seconds
Bad config deploy	rejected by `nginx -t`, auto-rollback, 0 failed requests
Nginx master crash	sub-second gap, automatic restart (production: VIP failover)
Traffic flood	attacker limited to 20 req/s + burst; other users unaffected
Certificate rotation	0 failed requests
Production configs	nginx -t, keepalived -t, promtool: all valid

> **Important:** During testing, stopping a backend made its host name resolve through the corporate DNS to public IP addresses, so Nginx tried to send requests (with the internal token) to the internet. The fix was to address backends by fixed private IP; test T28b guards against a regression. Details are in `docs/test-report.md`.